



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
UTM Johor Bahru

SECJ 3032: Software Engineering FYP1

Semester 01, 2024/2025

Software Design Specification (SDS)

LexiFlow:

A Web Platform for Translating and Transcribing
Native Mandarin Content

Version 1.0

Prepared by: ADAM ISMAIL HASSAN AMER

ABOURAYA

1. Revision Page

a. Overview

This version of the document encompasses the Software Design of LexiFlow. It provides a comprehensive description of the system's design, including the System Architectural Design, Detailed Description of Component, Data Design and User Interface Design.

b. Issuing Organization / Team

- Mandarin Language Learners (Students at UTM and other institutions)
- Language Educators (UTM Language Academy lecturers and instructors)

c. Authorship / Project Team Members

Team Member	Assigned Section	Status
Adam Abouraya	All	26/06/2025

d. Revision History

REVISION NO.	ISSUE DATE	DETAILS OF REVISION
01	26/06/2025	Initial Submission

Note:

This Software Design Descriptions (SDD template is adapted from IEEE Recommended Practice based on Software Design Descriptions (SDD) (IEEE Std. 10161998 1), that are simplified and customized to meet the need of SECJ3203 FYP1 SE course at Faculty of Computing, UTM. Examples of models are from Arlow and Neustadt (2002) and other sources stated accordingly.

Table of Contents

1	Frontispiece	1
2	Introduction	3
2.1	Purpose	3
2.2	Scope	3
2.3	Context	3
2.4	Summary	3
3	References	
4	Glossary	
5	Design Body	
5.1	Design Stakeholders and Their Concerns	
5.2	Context Viewpoint	
5.2.1	Design Concerns	
5.2.2	Design View (Use Case Diagram)	
5.3	Composition Viewpoint	
5.3.1	Design Concerns	
5.3.2	Design View (Architecture Style & Pattern Diagram)	
5.4	Logical Viewpoint	
5.4.1	Design Concerns	
5.4.2	Design View (Class Diagram)	
5.5	Information Viewpoint	
5.5.1	Design Concerns	
5.5.2	Design View (ERD Diagram)	
5.6	Interface Viewpoint	
5.6.1	Design Concerns	

- 5.6.2 Design View (Interfaces)
- 5.7 Structure Viewpoint
 - 5.7.1 Design Concerns
 - 5.7.2 Design View (Package Diagram)
- 5.8 Interaction Viewpoint
 - 5.8.1 Design Concerns
 - 5.8.2 Design View (Sequence Diagram)
- 5.9 State Dynamic Viewpoint
 - 5.9.1 Design Concerns
 - 5.9.2 Design View (State Machine Diagram, Storyboards / Interface-flow Diagram)
- 5.10 Algorithm Viewpoint
 - 5.10.1 Design Concerns
 - 5.10.2 Design View (Activity Diagram / Decision Tables)
- 6 User Interface Design**
- 7 Appendices**

2. Introduction

2.1 Purpose

This Software Design Document (SDD) describes the architecture and detailed design of the LexiFlow application. This document outlines the system's architectural design, including its major components, their interactions, and the data flow between them. It also provides detailed design specifications for each component, ensuring that the development team has a clear and comprehensive understanding of the system's structure and functionality.

2.2 Scope

The proposed platform is designed to process long-form audio or video content, accepting inputs up to 120 minutes from user-uploaded files or YouTube URLs. It will employ voice activity detection (VAD) and end-of-sentence detection to segment the audio into linguistically meaningful, sentence-level chunks. For transcription, the platform will utilize automatic speech recognition (ASR), and translations will be generated via a Neural Machine Transformer (NMT) model. The output will be synchronized, timestamped bilingual subtitles, featuring original language text, target-language translations, and pinyin annotations, all produced by the transformer model. To facilitate efficient reuse, all processed subtitles will be stored in a searchable database, indexed by content identifiers like URLs, eliminating the need for reprocessing. Users will interact with the platform through a web interface, which allows for content uploads and customization of subtitle outputs.

2.3 Context

This Software Design Document (SDD) serves as a critical guide for technical and managerial stakeholders involved in the design, development, and maintenance of the LexiFlow system. It is intended to be used by:

- a) Technical Teams: Including software architects, developers, and quality assurance personnel, as a comprehensive reference for implementing the system's functionalities and understanding its underlying architecture.

b) Designers: Providing guidance in the selection, organization, and presentation of design information, ensuring consistency and clarity in all design-related artifacts.

c) Project Managers and Stakeholders: To facilitate effective oversight, decision-making, and communication regarding the system's design and its adherence to specified requirements.

This document guides designers in the selection, organization, and presentation of design information for the LexiFlow project. For an organization developing its own SDD practices, the use of this standard promotes consistency and clarity across all design documentation.

2.4 **Summary**

Give an overview of the content of this SDS document.

3. **References**

Specify complete list of references using a standardized reference format.

4. Glossary

Definitions of all terms, acronyms and abbreviation used are to be defined here.

- UTM – Universiti Teknologi Malaysia
- UI – User Interface
- API – Application Programming Interface
- SRS – Software Requirement Specification
- ASR – Automatic Speech Recognition
- NMT – Neural Machine Transformer
- VAD – Voice Activity Detection

5. Design Body

5.1 Design Stakeholders and Their Concerns

The design stakeholders for the LexiFlow system were identified during the requirements gathering phase through interviews and surveys conducted with the UTM Language Academy. The primary design stakeholders include language learners (students), language educators (lecturers), system administrators, and the software development team.

a) Language learners are concerned with ease of use, accuracy of transcriptions and translations, and the availability of engaging and authentic learning materials. They require that the system delivers synchronized, accurate, and readable subtitles with pinyin annotations to support learning.

b) Language educators focus on the pedagogical quality of the content and tools that support learner progress, such as vocabulary saving, flashcards, and quizzes. They are concerned that the system enables effective teaching by providing reliable translations, customizable learning pathways, and ways to monitor and guide students.

c) System administrators are concerned with the stability, performance, and maintainability of the platform. They require clear documentation, secure access, and efficient management of content and user data.

d) The software development team focuses on delivering a robust and scalable architecture that meets stakeholder needs. They are concerned with ensuring the modularity of the system, ease of integration of third-party services, and future extensibility.

5.2 Context Viewpoint

5.2.1 Design Concerns

The purpose of the Context Viewpoint for LexiFlow is to clearly define the system's boundaries and interactions within its operational environment. LexiFlow is designed to provide an efficient and user-friendly platform for processing authentic Mandarin content and delivering it as synchronized bilingual subtitles with pinyin support. The

system enables language learners to upload or link to media content, access accurate transcriptions and translations, manage vocabulary, and enhance their language proficiency through interactive learning features. The main actors of the system include students (language learners), lecturers (language educators), system administrators, and external services such as YouTube Data API.

- a) Students (Language Learners):
 - Registering and logging into the system
 - Uploading audio/video content or submitting YouTube URLs for processing
 - Viewing synchronized bilingual subtitles with pinyin annotations
 - Searching word definitions and saving vocabulary for later review
 - Generating and practicing flashcards and quizzes from saved vocabulary
 - Bookmarking video segments for reference
 - Reporting translation errors collaboratively
- b) Stakeholders (Lecturers / Language Educators):
 - Uploading or managing curated learning materials for students
 - Monitoring student vocabulary progress and content interaction
 - Creating themed playlists to support structured learning sessions
- c) External Services:
 - YouTube Data API for fetching video metadata and captions

5.2.2 Design View

The system features of LexiFlow are organized into four primary functional modules: the User Module, Content Processing Module, Interactive Learning Module, and Assessment Module. These modules work together to deliver a comprehensive and engaging Mandarin language learning experience for users.

The User Module enables new users, including both students and lecturers, to register and create personal accounts on the platform. Once registered, users can securely log in to access the system's features. The module also allows users to manage their personal profiles, including updating learning preferences and reviewing activity history.

The Content Processing Module enables users to upload local media or submit YouTube URLs for processing. The system generates synchronized bilingual subtitles with Mandarin text, pinyin, and English translation. Users can also search and access previously processed content.

The Interactive Learning Module supports vocabulary learning. Users can search word definitions, save vocabulary, bookmark video segments, and report translation errors to improve content quality.

The Assessment Module allows users to review saved vocabulary. The system can automatically generate flashcards from the user's saved vocabulary. Furthermore, the platform offers automatically generated quizzes based on the user's saved vocabulary.

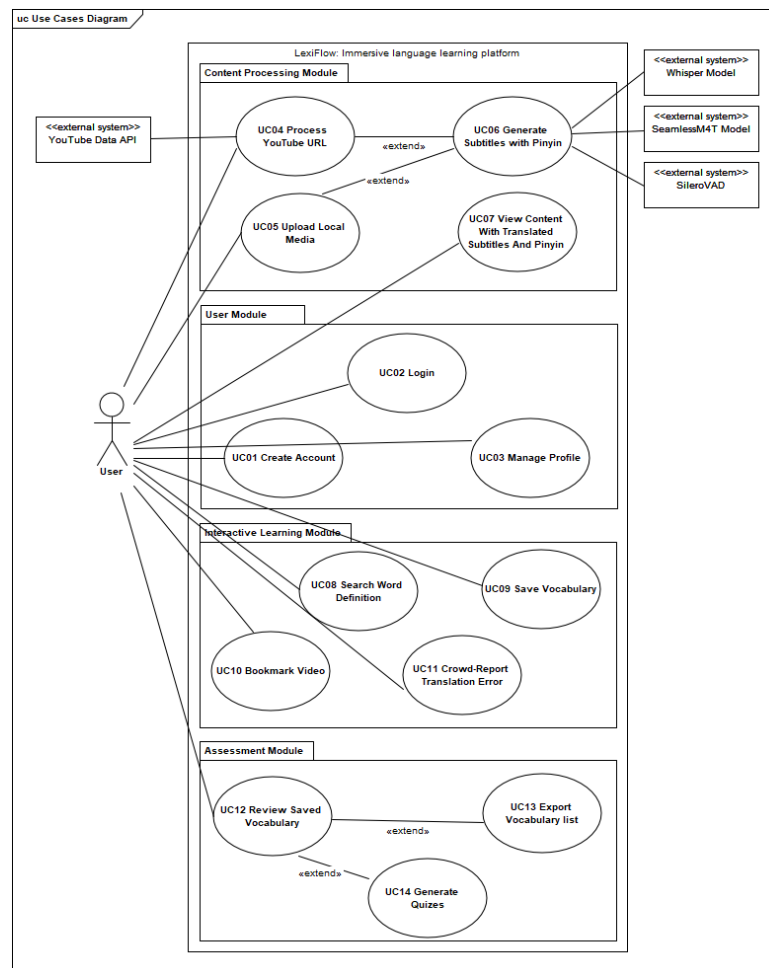


Figure 2.1: Use Case Diagram for <LexiFlow>

5.3 Composition Viewpoint

5.3.1 Design Concerns

The composition viewpoint focuses on how the LexiFlow system is organized as a series of modular components connected through a Pipe and Filter architecture. This structure enables clear separation of responsibilities between processing stages, improves scalability, and allows individual components to be updated or replaced independently. The design consists of the following major components:

- a) Design components:
 - User Interface (UI): The Vue.js web application that allows users to interact with the system — uploading content, viewing subtitles, managing vocabulary, and taking quizzes.
 - Backend Services: Node.js and FastAPI components that coordinate requests from the frontend, manage user sessions, and control the processing pipeline.
 - Audio Processing: A sequence of processing filters — Silero VAD (segmentation), Whisper ASR (transcription), SeamlessM4Tv2 (translation), and PyPinyin (pinyin annotation) — which transform audio/video input into educational outputs.
 - Database: MongoDB for persistent storage of user profiles, processed subtitles, content metadata, and vocabulary lists.
 - External APIs and Models: YouTube Data API for metadata retrieval.
- b) Subsystems:
 - User Management Module: Manages user registration, login, profile updates, and authentication.
 - Content Processing Module: Manages content upload, YouTube URL processing, and subtitle generation.
 - Interactive Learning Module: Provides bookmarking, vocabulary saving, and error reporting features.
 - Assessment Module: Provides flashcards, quizzes and vocabulary lists.

5.3.2 Design View

The component model is developed using a Pipe and Filter approach, where each processing step is a self-contained filter that performs a specific transformation on the content. Data flows through these filters in sequence, with orchestration handled by backend services. Figure 5.1 illustrates how components are connected and how the processing pipeline operates within the larger system.

The User Interface allows learners and educators to upload content, manage learning tools, and view synchronized bilingual subtitles. The Backend Services coordinate user interactions with the processing pipeline and database. These services queue tasks, invoke each filter in sequence, and manage the data flow between components.

From the youtube link the audio file is extracted using Youtube API. Input audio is segmented into audio chunks using Silero VAD, transcribed with Whisper ASR into source text, translated to English with SeamlessM4Tv2, and annotated with pinyin using PyPinyin. Each stage passes its output to the next filter, enabling efficient and accurate content processing.

The Database stores processed subtitles, vocabulary lists, user profiles, and metadata. It supports fast retrieval and reuse of previously processed materials. External APIs provide specialized processing capabilities, such as accessing YouTube video metadata.

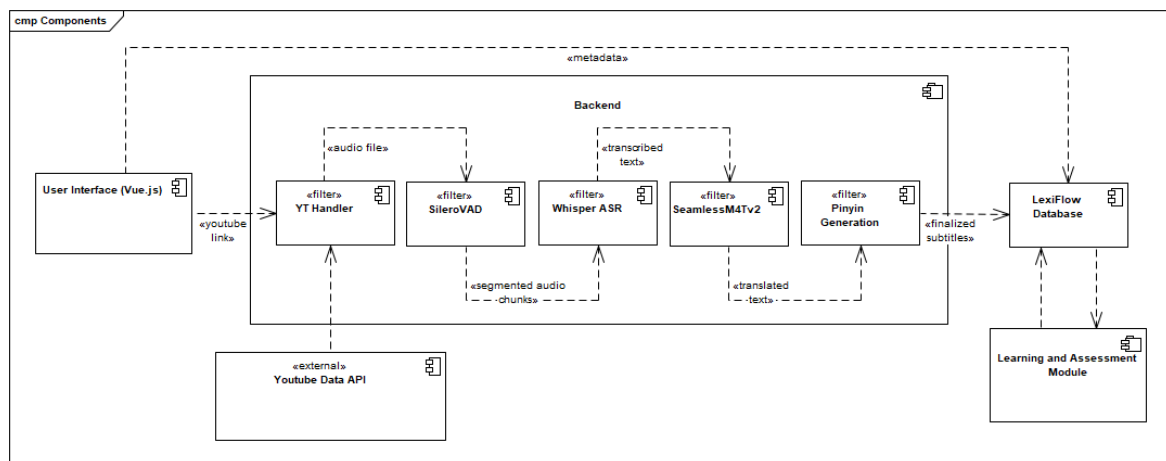


Figure 3.1: Component Diagram of <LexiFlow>

To make the component-level view above concrete, Figure 3.2 presents the system architecture in terms of the final implementation described in Chapter 5 of the accompanying thesis: a React 18/TypeScript single-page frontend; a Node.js (Express) service handling business logic, authentication, and orchestration; a Python (FastAPI) service running the AI media-processing pipeline as a chain of filters (FFmpeg normalisation, Whisper Large v3 Turbo transcription, LLM translation, and PyPinyin/HSK annotation); a PostgreSQL database for relational application data; a

MongoDB store for the job queue and generated subtitle documents; and externally hosted AI inference services reached over HTTPS.

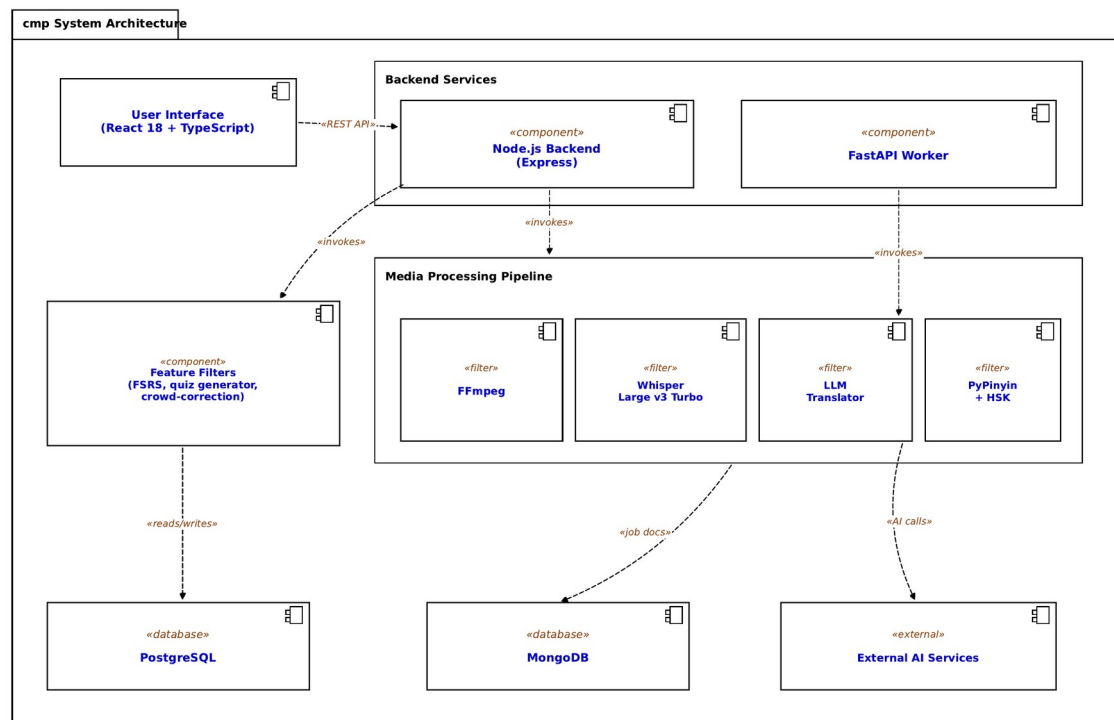


Figure 3.2: System Architecture Diagram of <LexiFlow>

5.4 Deployment Viewpoint

5.4.1 Design Concerns

The Deployment Viewpoint describes the physical distribution of LexiFlow's software components across runtime environments and how they communicate at runtime. Its purpose is to make explicit which components run where, how they are packaged and deployed, and what infrastructure dependencies the system has, so that stakeholders can reason about availability, scalability, and operating cost.

5.4.2 Design View

Figure 3.3 shows where each component physically runs at deployment time. The frontend is served as a static build from Vercel's edge content delivery network (CDN). The Node.js backend, the FastAPI worker, and PostgreSQL are co-located on a single Tencent Cloud Lighthouse virtual machine via Docker Compose, with a SaladPool instance kept available as a rarely used failover worker. MongoDB Atlas is hosted

separately as a managed service. GitHub Actions builds and pushes a container image to GitHub Container Registry (GHCR) on every push to the main branch and redeploys the virtual machine automatically, so no manual deployment step is required. Because all AI inference (transcription, translation, quiz grading, and segment correction) is delegated to externally hosted language models, the deployment itself requires no GPU.

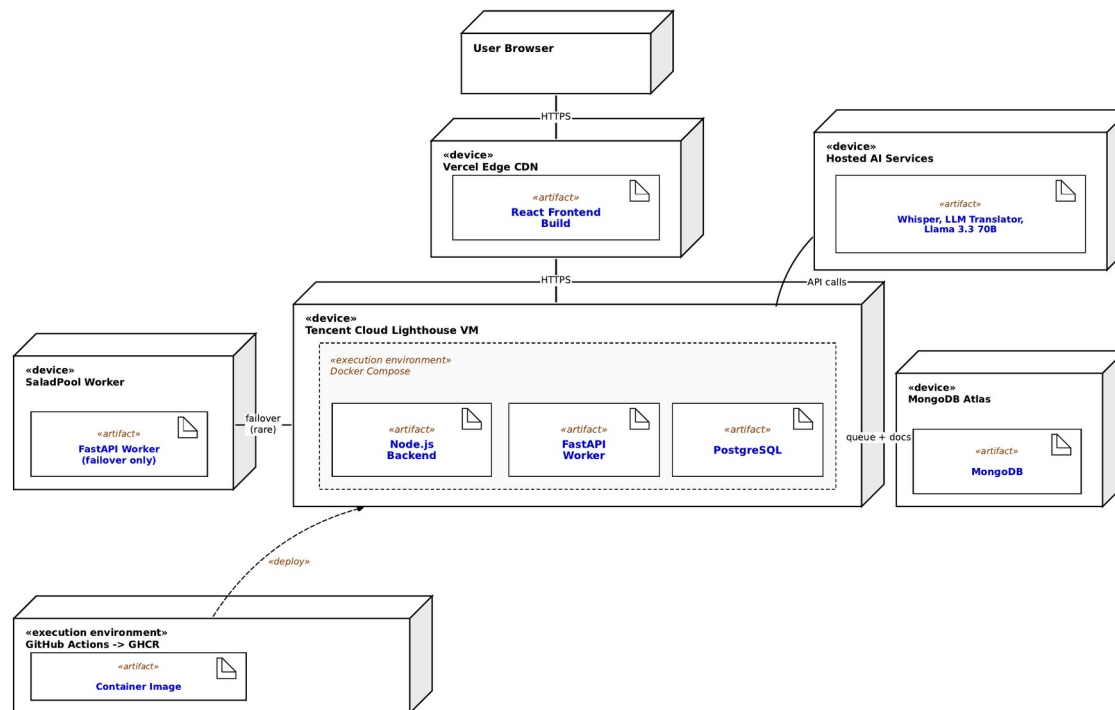


Figure 3.3: Deployment Diagram of <LexiFlow>

5.5 Logical Viewpoint

5.5.1 Design Concerns

The purpose of the Logical Viewpoint is to elaborate the key classes and their relationships within the LexiFlow system. The system implements a modular design where users, content, learning progress, and processed segments are modeled through distinct classes and associations.

a) Classes and Interfaces

- **User:** Represents all users of the system, including students and lecturers. Attributes include `userID`, `username`, `email`, `password`, and

registrationDate. Provides interfaces for user authentication and profile management.

- Educator / LanguageLearner: Subclasses of User. Educator adds attributes such as department and institution. LanguageLearner includes proficiencyLevel, preferredLanguage, and learningGoals, supporting personalized learning paths.
- Content: Represents uploaded or linked media content (audio/video). Attributes include contentID, title, duration, sourceURL, and uploadDate. Interfaces manage content submission and tracking.
- LearningProgress: Tracks user learning activity with attributes such as progressID, userID, contentID, completionStatus, and wordsReviewedCount. Supports learning analytics and progress monitoring.
- Segment: Defines segmented portions of audio/video content with segmentID, startTime, endTime. Segments act as the basic units for transcription and translation.
- Transcription: Stores Mandarin text transcribed from each segment. Attributes include transcriptionID, text, and generationDate.
- Translation: Holds the English translation for each segment. Attributes include translationID, text, and generationDate.
- Subtitle: Combines transcription, translation, and pinyin annotations into a subtitle object. Attributes include subtitleID, originalTextSegment, translatedTextSegment, pinyinAnnotation, startTime, and endTime. Supports synchronized playback and learning.

b) Relationships

- User and Educator / LanguageLearner: The User class has a one-to-one inheritance relationship with Educator and LanguageLearner. This supports role-specific functionality while maintaining a unified user system.
- User and LearningProgress: A user can have multiple learning progress records, creating a one-to-many relationship between User and LearningProgress. This tracks progress across various content items.

- User and Content: Users (mainly educators or learners) upload content. The relationship between User and Content is one-to-many, where one user can upload multiple pieces of content.
- Content and Segment: Each content item is divided into multiple segments. The relationship between Content and Segment is one-to-many.
- Segment and Transcription / Translation / Subtitle: Each segment is associated with one Transcription, one Translation, and one Subtitle, forming one-to-one relationships between Segment and these related classes.
- LearningProgress and Content: Each LearningProgress record is linked to a specific piece of content, enabling precise tracking of what the learner has engaged with. The relationship is many-to-one between LearningProgress and Content.

5.5.2 Design View

a) Subsystem < User Management >

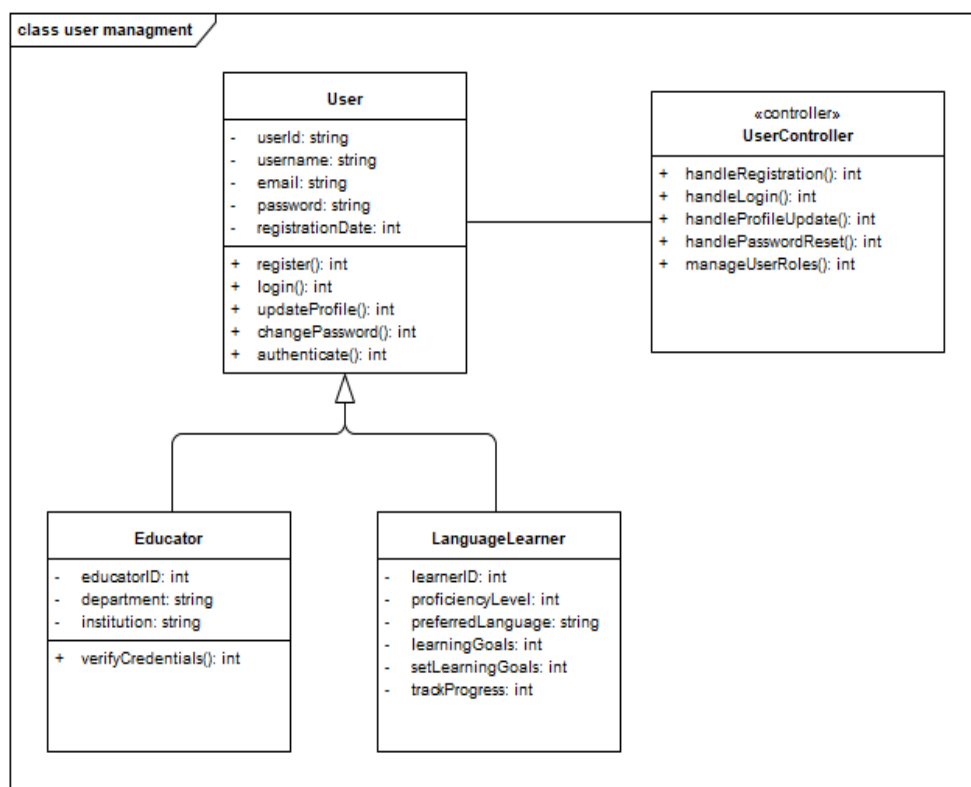


Figure 4.1: Class diagram for <User Managment> Subsystem

b) Subsystem < Content Processing >

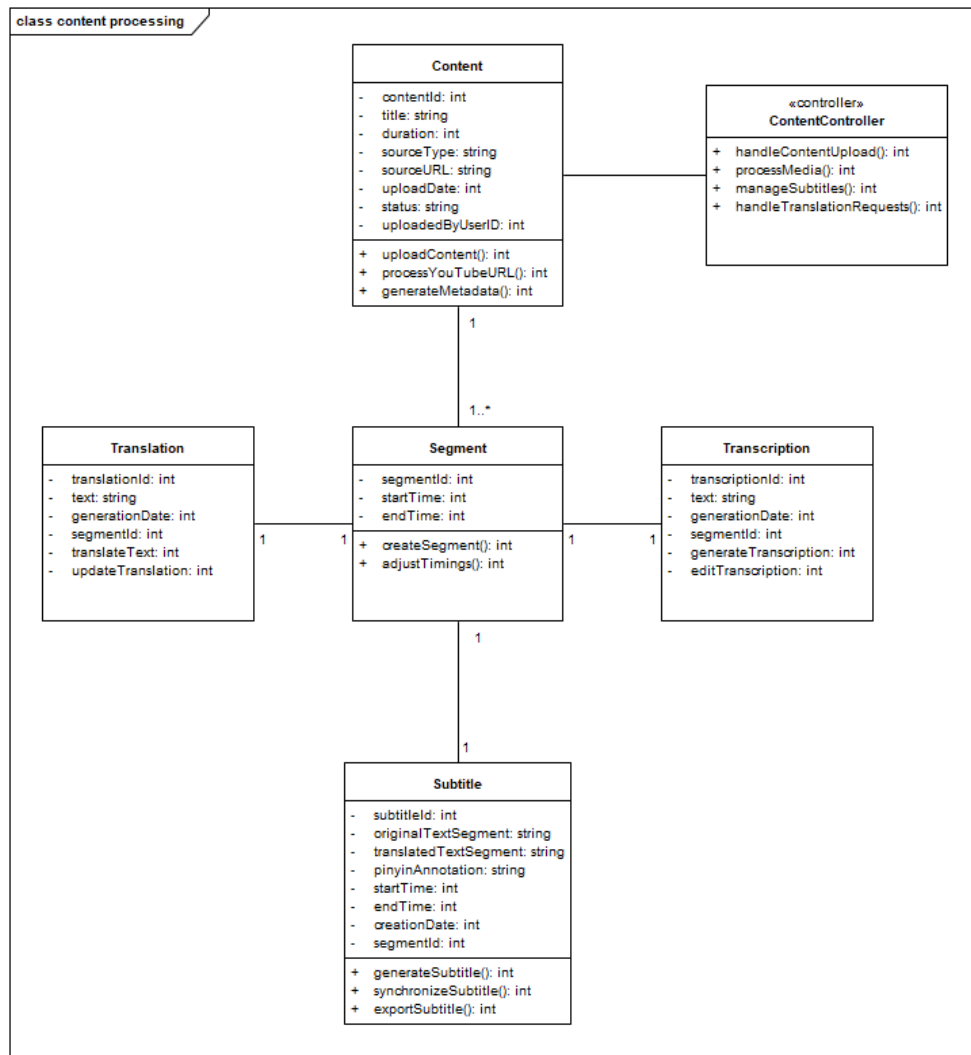


Figure 4.2: Class diagram for < Content Processing > Subsystem

c) Subsystem <Interactive Learning>

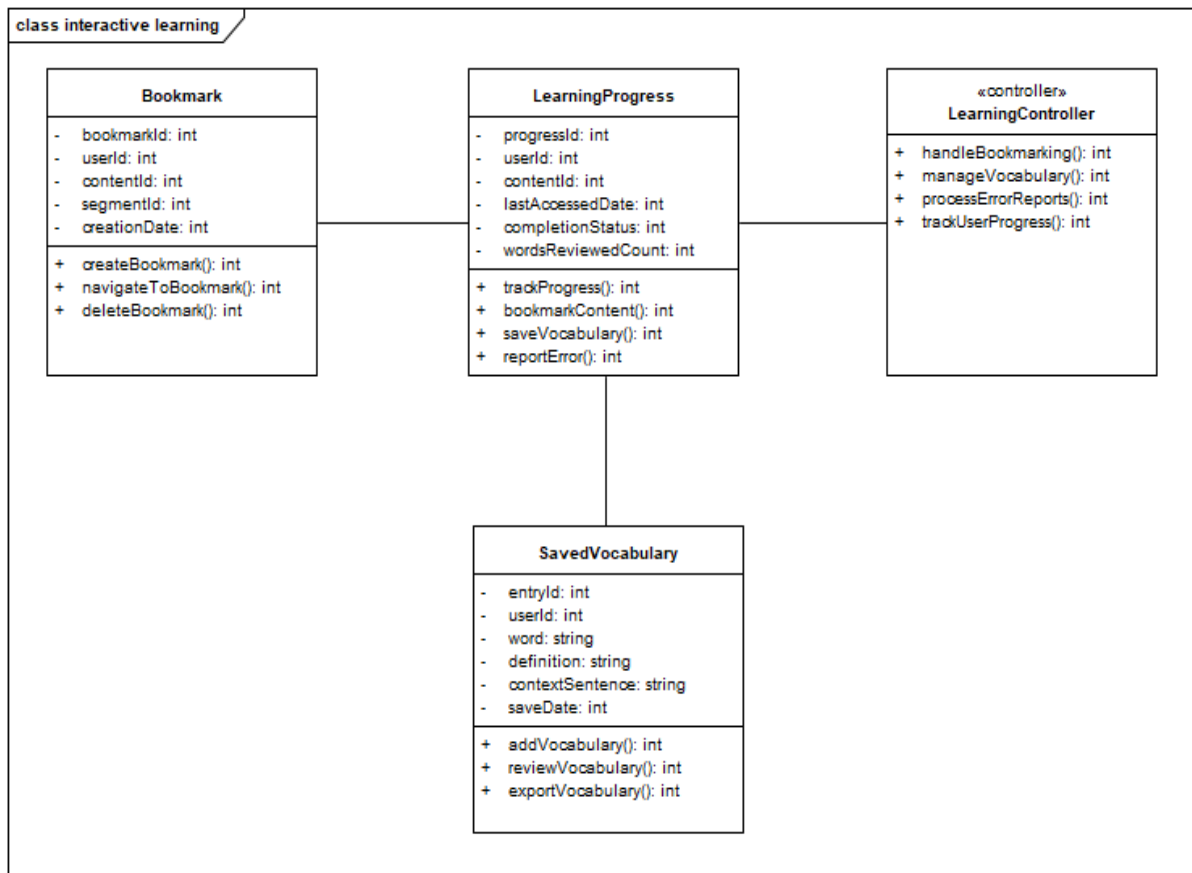


Figure 4.3: Class diagram for <Interactive Learning> Subsystem

d) Subsystem <Assessment>

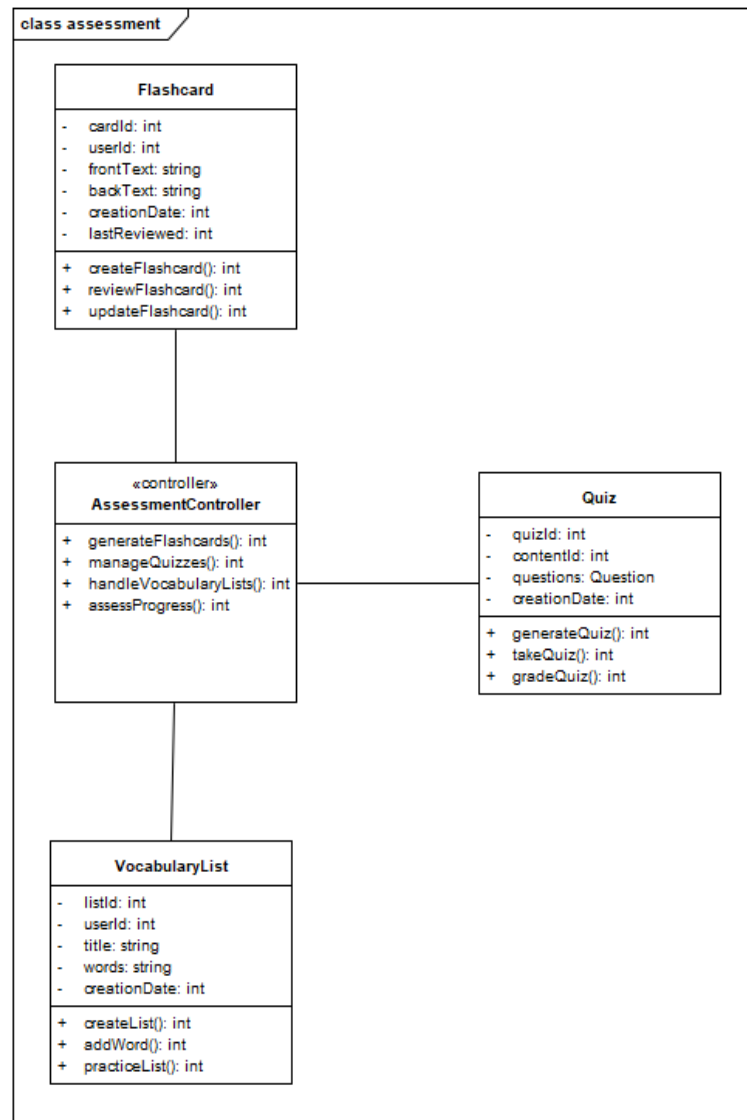


Figure 4.4: Class diagram for <Assessment> Subsystem

5.6 Information Viewpoint

5.6.1 Design Concerns

The major data or systems entities are stored into a relational database named as... comprises n entities ...

Entity Name	Description
User	Stores general details of all users, including educators and language learners.
Educator	Represents users who are educators, inheriting general user properties and having specific attributes related to their educational background.

LanguageLearner	Represents users who are language learners, inheriting general user properties and having specific attributes related to their learning preferences.
Content	Holds information about various types of uploaded content, such as videos or articles, including their source and upload details.
Segment	Divides content into smaller, manageable parts, defining specific start and end times within a piece of content.
Transcription	Stores the transcribed text for specific segments of content, along with the date of generation.
Translation	Contains the translated text for specific content segments, including the generation date.
Subtitle	Stores subtitle information, including original and translated text, phonetic annotations, and timing for specific content segments.
LearningProgress	Tracks a user's progress through content, including access dates, completion status, and words reviewed.
Bookmark	Allows users to save specific segments within content for later reference.
SavedVocabulary	Stores vocabulary words saved by users, along with their definitions, context, and save dates.
Flashcard	Represents digital flashcards created by users for vocabulary practice.
Quiz	Contains details about quizzes related to content, including questions and creation date.
VocabularyList	Stores custom vocabulary lists created by users, including a title and associated words.

5.6.2 Design View

Data Dictionary

Entity: User

Attribute Name	Type	Description
userId	INT	Unique identifier for each user (PK)
username	VARCHAR	User's chosen username
email	VARCHAR	User's email address
password	VARCHAR	Hashed password for user authentication
registrationDate	DATETIME	Date and time when the user registered

Entity: < Educator>

Attribute Name	Type	Description
userId	INT	Unique identifier for the educator (PK, FK to User)
education	VARCHAR	Educational background of the educator
department	VARCHAR	Department the educator belongs to
institution	VARCHAR	Institution where the educator is affiliated

Entity: < LanguageLearner>

Attribute Name	Type	Description
learnerID	INT	Unique identifier specific to the learner role
proficiencyLevel	VARCHAR	Learner's current language proficiency level
preferredLanguage	VARCHAR	The language the learner prefers to study
learningGoals	TEXT	Specific learning objectives of the learner

Entity: < Content>

Attribute Name	Type	Description
contentId	INT	Unique identifier for each content item (PK)
title	VARCHAR	Title of the content
duration	INT	Duration of the content in seconds
sourceType	VARCHAR	Type of content source (e.g., "video", "article")
sourceURL	VARCHAR	URL of the original content source
uploadDate	DATETIME	Date and time when the content was uploaded
uploadedByUserID	INT	ID of the user who uploaded the content (FK to User)

Entity: < Segment>

Attribute Name	Type	Description
segmentId	INT	Unique identifier for each segment (PK)
contentId	INT	Identifier of the content this segment belongs to (FK to Content)
startTime	TIME	Start time of the segment within the content

endTime	TIME	End time of the segment within the content
---------	------	--

Entity: < Transcription>

Attribute Name	Type	Description
transcriptionId	INT	Unique identifier for each transcription (PK)
text	TEXT	The transcribed text
generationDate	DATETIME	Date and time when the transcription was generated
segmentId	INT	Identifier of the segment transcribed (FK to Segment)

Entity: < Translation>

Attribute Name	Type	Description
translationId	INT	Unique identifier for each translation (PK)
text	TEXT	The translated text
generationDate	DATETIME	Date and time when the translation was generated
segmentId	INT	Identifier of the segment translated (FK to Segment)

Entity: < Subtitle>

Attribute Name	Type	Description
subtitleId	INT	Unique identifier for each subtitle (PK)
originalTextSegment	TEXT	The original text for the subtitle segment
translatedTextSegment	TEXT	The translated text for the subtitle segment
phoneticAnnotation	VARCHAR	Phonetic annotation for the text (e.g., pronunciation guide)
startTime	TIME	Start time of the subtitle display
endTime	TIME	End time of the subtitle display
creationDate	DATETIME	Date and time when the subtitle was created
segmentId	INT	Identifier of the segment the subtitle belongs to (FK to Segment)

Entity: < LearningProgress>

Attribute Name	Type	Description
progressId	INT	Unique identifier for each progress entry

		(PK)
userId	INT	Identifier of the user (FK to User)
contentId	INT	Identifier of the content being tracked (FK to Content)
linkAccessedDate	DATETIME	Date and time when the content was last accessed
completionStatus	BOOLEAN	Indicates if the content has been completed

Entity: < Bookmark>

Attribute Name	Type	Description
bookmarkId	INT	Unique identifier for each bookmark (PK)
userId	INT	Identifier of the user who created the bookmark (FK to User)
contentId	INT	Identifier of the content bookmarked (FK to Content)
segmentId	INT	Identifier of the segment bookmarked (FK to Segment)
creationDate	DATETIME	Date and time when the bookmark was created

Entity: < SavedVocabulary>

Attribute Name	Type	Description
entryId	INT	Unique identifier for each saved vocabulary entry (PK)
userId	INT	Identifier of the user who saved the word (FK to User)
word	VARCHAR	The vocabulary word saved
definition	TEXT	Definition of the saved word
contextSentence	TEXT	Sentence providing context for the word
saveDate	DATETIME	Date and time when the word was saved

Entity: < Flashcard>

Attribute Name	Type	Description
userId	INT	Identifier of the user who created the flashcard (FK to User)
frontText	TEXT	Text displayed on the front of the flashcard
backText	TEXT	Text displayed on the back of the flashcard
creationDate	DATETIME	Date and time when the flashcard was

		created
lastReviewed	DATETIME	Date and time when the flashcard was last reviewed

Entity: < Quiz>

Attribute Name	Type	Description
quizId	INT	Unique identifier for each quiz (PK)
contentId	INT	Identifier of the content the quiz is related to (FK to Content)
questions	JSON	Quiz questions (can be a separate table)
creationDate		Date and time when the quiz was created

Entity: < VocabularyList>

Attribute Name	Type	Description
listId	INT	Unique identifier for each vocabulary list (PK)
userId	INT	Identifier of the user who created the list (FK to User)
title	VARCHAR	Title of the vocabulary list
words	JSON	Words included in the list (can be a separate table)
creationDate	DATETIME	Date and time when the list was created

Entity Relationship Diagram (ERD)

The Entity-Relationship Diagram (ERD) represents the data structure of the LexiFlow system, illustrating the attributes of its main entities and relationships between them.

- a) Entities
 - User: Represents a platform user with a unique userId, login credentials, and profile information. It is the base entity for all user types.
 - Educator: A specialized type of User who can upload educational content. Includes additional details such as education, department, and institution.
 - LanguageLearner: A specialized type of User focused on learning languages. Tracks proficiency level, preferred language, and learning goals.
 - Content: Stores information about uploaded educational media (videos, documents, etc.), including metadata like title, duration, and upload source.

- Segment: Divides Content into smaller parts (segments), defined by startTime and endTime, to support fine-grained transcription, translation, and subtitle creation.
- Transcription: Contains text transcriptions for a specific Segment, generated to assist learners in reading along with the audio/video content.
- Translation: Holds the translated text for a specific Segment, enabling multilingual learning and comprehension.
- Subtitle: Stores subtitles for a Segment, including original and translated text with phonetic annotations and precise timing for playback synchronization.
- LearningProgress: Tracks the learner's progress on specific content, including completion status and vocabulary review statistics.
- Bookmark: Allows users to save specific segments of content for easy navigation and review later.
- SavedVocabulary: Maintains vocabulary words saved by the user, with definitions and example contexts to aid memorization.
- Flashcard: Digital flashcards created by users to practice vocabulary or concepts, with fields for front and back text and review history.
- Quiz: Represents quizzes generated from content, storing questions and associated metadata to assess user learning.
- VocabularyList: User-curated lists of vocabulary words, which can be organized and practiced systematically.

b) Relationships

- User and LearningProgress: One-to-many relationship
- User and Content (uploaded content): One-to-many relationship
- Content and Segment: One-to-many relationship
- Segment and Transcription: One-to-one relationship
- Segment and Translation: One-to-one relationship
- Segment and Subtitle: One-to-one relationship
- User and Bookmark: One-to-many relationship
- User and SavedVocabulary: One-to-many relationship
- User and Flashcard: One-to-many relationship
- User and VocabularyList: One-to-many relationship

- Content and Quiz: One-to-many relationship

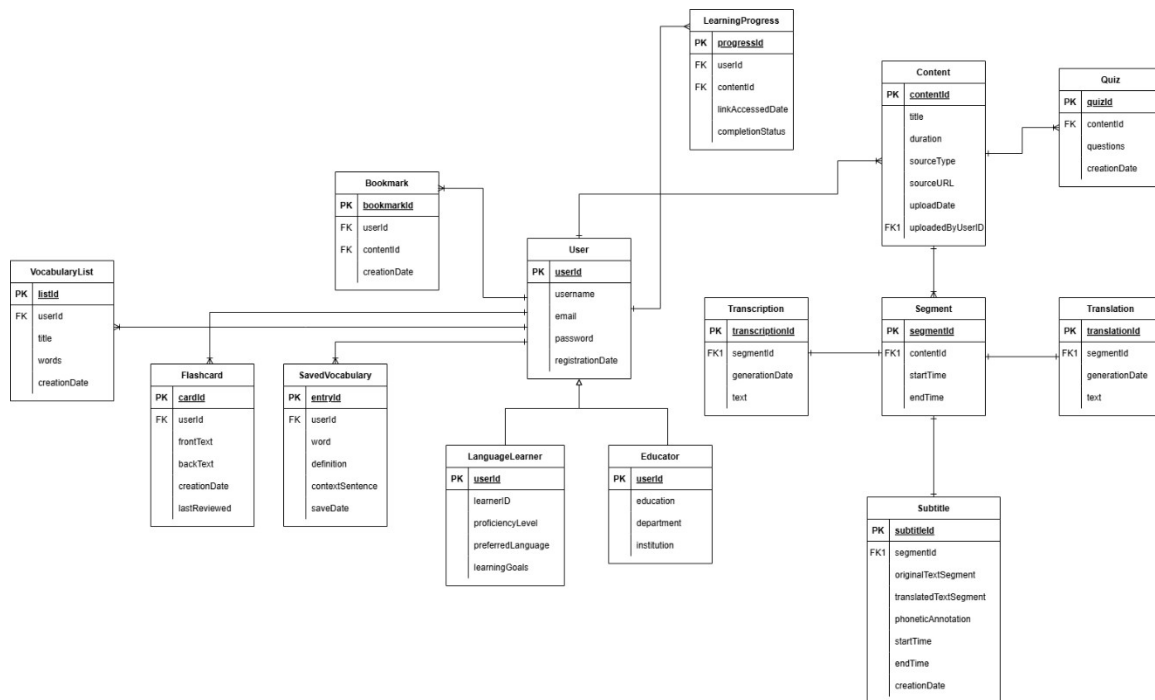


Figure 5.1: ERD diagram for LexiFlow System

5.7 Interface Viewpoint

5.7.1 Design Concerns

The interface viewpoint describes how the LexiFlow system will interact with its users, underlying hardware, integrated software components, and various communication channels. It covers the multiple interfaces that facilitate these interactions, including user interfaces, hardware interfaces, software interfaces, and communication interfaces.

The goal of this viewpoint is to provide a comprehensive understanding of how users will interact with the system's features and how LexiFlow interfaces with external components and services, ensuring seamless integration and robust operation. This description will be invaluable for designers, programmers, and testers, guaranteeing that the correct services are provided and that users can efficiently complete all expected features.

5.7.2 Design View

a) Logical Characteristics

- UC01: Create Account The interface shall include input fields for full name, email, password, confirm password, and a selection for user type (e.g., Language Learner, Educator). Error validation will display appropriate prompts if any required field is left empty, if the email format is invalid, or if the passwords do not match or meet complexity requirements.
- UC02: Login This interface includes input fields for email/username and password, and a prominent login button. If login fails due to invalid credentials, a message will appear indicating the incorrect input. Successful login redirects the user to their personalized dashboard, providing access to their content and learning tools.
- UC03: Manage Profile The interface allows users to update personal details such as name and email, and to change their account password. Changes are designed to be reflected immediately upon successful submission and stored securely in the system's database.
- UC04: Process YouTube URL The interface will feature an input field for pasting YouTube video URLs and a clear "Process" or "Submit" button. Upon submission, the interface will provide visual feedback on the processing status

(e.g., "Processing initiated," "Generating subtitles"), and notifications upon completion.

- UC05: Upload Local Media This interface will provide a file upload mechanism, such as a "Browse" button to select local audio or video files and potentially a drag-and-drop area. It will display the selected file's name and upload progress, along with notifications for successful upload or any errors (e.g., unsupported format, file size limit).

- UC06: Generate Subtitles While largely a backend process, the user interface will display the status of subtitle generation for processed content (e.g., "Processing," "Subtitles Ready"). Once generated, it will provide options to view, download, or interact with the bilingual and pinyin-annotated subtitles.

- UC07: Find Translated Content The interface will include a search bar with filtering capabilities (e.g., by language, topic, upload date) to help users find previously processed and translated content. Search results will be displayed in a clear, browsable list with relevant content metadata.

- UC08: Search Word Definition Within the subtitle viewing interface, users will be able to interact with individual words (e.g., by clicking or hovering). This interaction will trigger a pop-up or side panel display showing the word's definition, pinyin pronunciation, and potentially example sentences, retrieved from an integrated dictionary.

- UC09: Save Vocabulary Integrated within the subtitle viewer or word definition pop-up, this interface element (e.g., a "Save" button or star icon) allows users to add selected words or phrases directly to their personal vocabulary list. Visual confirmation will indicate successful saving.

- UC10: Bookmark Video A clear "Bookmark" icon or button will be present on the video playback interface. Clicking it will allow users to save the current video (and potentially specific timestamps) to their personal collection for easy access and later review.

- UC11: Crowd Report Translation Error This interface will allow users to select a specific subtitle segment and initiate a "Report Error" function. A form

will appear, enabling the user to describe the inaccuracy and suggest corrections, contributing to community-driven content improvement.

- **UC12: Review Saved Vocabulary** This interface will present the user's saved vocabulary list in a browsable and potentially sortable format. Each entry will display the word, its pinyin, definition, and the context from which it was saved. Options for filtering or searching within the list will also be provided.
- **UCI3: Generate Flashcards** From the saved vocabulary list, this interface will allow users to select specific words or entire lists and generate digital flashcards. The interface will then display these flashcards for interactive practice, allowing users to "flip" cards to reveal definitions and self-assess.
- **UCI4: Export Vocabulary List** This interface provides options to export the saved vocabulary list. Users can select the desired file format (e.g., CSV, TXT) and specify fields to include before initiating a file download to their local device.
- **UCI5: Generate Quizzes** This interface enables users to configure and generate quizzes based on their saved vocabulary or processed content. Users will select quiz parameters such as question type, number of questions, and source material. The generated quiz will then be presented in an interactive format for completion and immediate feedback.

b) Interface Optimization

Actor	User Interface Accessible Use Cases	Description
Language Learner	UC01, UC02, UC03, UC04, UC05, UC06, UC07, UC08, UC09, UC10, UC11, UC12, UCI3, UCI4, UCI5	Language Learners can create/login, manage profile, upload/process content, view/find translated content, utilize learning tools like word definitions, vocabulary saving, flashcards, quizzes, and contribute feedback on translations.

Educator	UC01, UC02, UC03, UC04, UC05, UC06 (status/view), UC07, UC11, UCI4 (for curriculum prep)	Educators can create/login, manage profile, upload/process content, view/find translated content, export vocabulary (potentially for teaching materials), and contribute feedback on translations. Their focus is on content creation and access for educational purposes.
----------	--	--

Hardware Interfaces

The LexiFlow system primarily operates as a web-based application, accessible via various client devices such as mobile devices (Android and iOS platforms) and laptop/desktop systems. Development and administrative tasks also rely on standard computing hardware. The system requires stable network connectivity for real-time data synchronization with cloud services. The following describes the logical characteristics and specifications for these hardware components to ensure efficient operation.

Hardware Component	Specification	Purpose
Mobile Device	Processor: ARM Cortex-A53 or higher	Ensure smooth operation and responsiveness of the web application on mobile browsers.
	RAM: 4GB or higher	Support multitasking and provide a good user experience when accessing LexiFlow.
	Storage: 64GB or higher	Accommodate mobile browser cache and temporary data, not direct application installation.

	Network Connection: Wi-Fi or 4G LTE	Real-time data transfer for content Browse, content submission (URLs), and interaction with learning tools.
	Operating System: Android 8.0 or higher, iOS 12.0 or higher	Ensure compatibility with modern mobile browser features and smooth web application performance.
Laptop /Desktop	Processor: Intel Core i5 or higher, AMD Ryzen 5 or higher	Required for development and administrative tasks related to LexiFlow, and for users accessing the web application from desktops.
	RAM: 4GB or higher	Efficient multitasking and smooth operation of web browsers, development tools, and administrative interfaces.
	Storage: 256GB SSD or higher	Sufficient storage for development environment, tools (e.g., IDEs, Git), and local caching of web application data.
	Network Connection: Wi-Fi 4 or higher, Ethernet	Stable and high-speed connection to interact with cloud services (for both users and developers/admins) and for web development and management tasks.
Network	Wi-Fi: Wi-Fi 4 or higher, 4G LTE	Provides continuous communication with the cloud-based backend services (e.g., for content processing updates, vocabulary synchronization) and external APIs.

Software Interfaces

The LexiFlow web application relies on a combination of modern development tools, frameworks, and cloud services to deliver a responsive, scalable, and intelligent language learning experience. These software products provide functionality for code

development, user interface building, authentication, data management, and AI-driven user interaction.

Software Product	Specification	Purpose
IDE	Name: Visual Studio Code	Primary environment for writing, debugging, and running the code for the LexiFlow web application (frontend and backend development).
	Version: 1.60.0 or higher	
	Source: https://code.visualstudio.com/	
Version Control System	Name: Git	Manages source code versioning, enabling collaborative development, tracking changes, and ensuring code integrity across the LexiFlow project.
	Version: 2.30.0 or higher	
	Source: https://git-scm.com/	
Frontend Framework	Name: Vue.js	Framework used for building the dynamic and interactive user interface of the LexiFlow web application, ensuring a responsive and intuitive user experience.
	Version: Latest stable release	
	Source: https://vuejs.org/	
Backend Language/ Framework	Name: Python	Primary language and framework for developing the LexiFlow backend logic, including content processing, API integrations, and database interactions.
	Version: Python 3.10	
	Source: https://www.python.org/	
Authentication	Name: Firebase Authentication	Manages user authentication (sign-up, login, password management) for LexiFlow accounts, integrating securely with the database.
	Version: Latest compatible SDK	
	Source: firebase.google.com/	
Database	Name: MongoDB	Stores and retrieves all application data,
	Version: Latest	

	compatible SDK	including user profiles, content metadata, subtitles, vocabulary lists, and quiz data, providing a scalable NoSQL cloud database.
	Source: https://www.mongodb.com/	

Communication Interfaces

The communication between the LexiFlow web application, its cloud-based backend services, and integrated external APIs is managed through multiple interface technologies, ensuring secure data transmission, reliable interaction, and seamless synchronization.

a) RESTful APIs

RESTful APIs are the primary mechanism for facilitating communication between the LexiFlow client application (frontend), its backend services, and external cloud-based AI services. They enable essential operations such as user registration and login, content upload initiation, Browse and retrieving processed content, saving vocabulary, and submitting error reports.

b) TCP/IP Protocol

TCP/IP forms the fundamental communication protocol for transmitting data across the internet between user devices, LexiFlow's backend, and cloud services. It ensures reliable and ordered data transfer, providing the backbone for maintaining secure and consistent connectivity required for all application functionalities, including streaming content processing and real-time data updates.

c) WebSocket (for real-time features)

For features requiring real-time updates and interactive synchronization, such as potentially displaying subtitles as they are being generated or dynamic quiz interactions, WebSocket protocol may be utilized. This provides a persistent, full-duplex communication channel over a single TCP connection, reducing latency compared to traditional HTTP requests.

5.8 Interaction Viewpoint

5.8.1 Design Concerns

The Interaction Viewpoint focuses on defining how various components within the LexiFlow system communicate and work together to fulfill user actions. This viewpoint outlines the interaction strategies between users, system modules, and external services. It is essential for understanding the flow of control and data during key activities, such as uploading content, processing subtitles, and interacting with vocabulary tools.

a) Interaction Flow

- The user accesses the Content Upload feature via the frontend web interface.
- The user uploads a local video or enters a YouTube URL.
- The frontend sends the upload request to the ContentController in the backend.
- The ContentController stores metadata in the database and triggers the PipelineProcessor.
- The PipelineProcessor runs the audio through Silero VAD to segment the audio.
- Each segment is transcribed using Whisper ASR.
- Transcriptions are passed to SeamlessM4Tv2 for English translation.
- Translations are then annotated with pinyin using PyPinyin.
- All output is compiled into subtitle entries and stored in the database.
- The frontend retrieves the generated subtitles and displays them to the user with synchronized playback.
- The user can now highlight words, save them, and generate flashcards or quizzes.

b) Why

- Users want to learn Mandarin from authentic content with accurate, bilingual subtitles.
- LexiFlow automates this process, turning real video/audio into structured, interactive learning materials without manual intervention.
- c) **Where?**
 - User: Interacts from the frontend web application to upload content and view results.
 - Frontend (UI Layer): Captures user actions and displays feedback (e.g., subtitles, vocabulary tools).
 - Backend (Logic Layer): Handles the processing logic, including pipeline control, content storage, and learning tools.
 - External Services: Includes Whisper, Silero VAD, SeamlessM4Tv2, and PyPinyin libraries that run AI-based processing on uploaded content.
- d) **How?**
 - The user submits media or a link through the frontend.
 - The frontend sends the request to the backend API.
 - The backend triggers the pipeline to segment, transcribe, translate, and annotate the content.
 - Subtitles are generated and stored in the database.
 - The frontend retrieves and displays the subtitles in a synchronized player.
 - The user interacts with subtitles to save vocabulary, generate quizzes, or review content.
- e) **Interaction Levels:**
 - **Entity-Level Interactions:**
 - i. The User interacts with the Vue.js UI to upload content and view subtitles.
 - ii. The UI interacts with backend controllers to request processing and fetch results.
 - iii. The Backend Controller invokes external processing modules to perform VAD, ASR, translation, and pinyin generation.
 - **Module-Level Interactions:**
 - i. The Frontend UI Layer communicates with the Content Processing Module to upload content and retrieve subtitles.
 - ii. The Content Controller communicates with the Pipeline Processing Module, which executes each filter (VAD, ASR, etc.).
 - iii. The Vocabulary Module interacts with the Database Layer to store and retrieve saved words, flashcards, and quizzes.
 - **External System Interaction:**
 - i. YouTube API fetches video metadata.
 - ii. Silero VAD, Whisper ASR, SeamlessM4Tv2 are called during pipeline execution to process the audio and generate subtitles and annotations.

5.8.2 Design View

a) SD001: Sequence diagram for Create Account

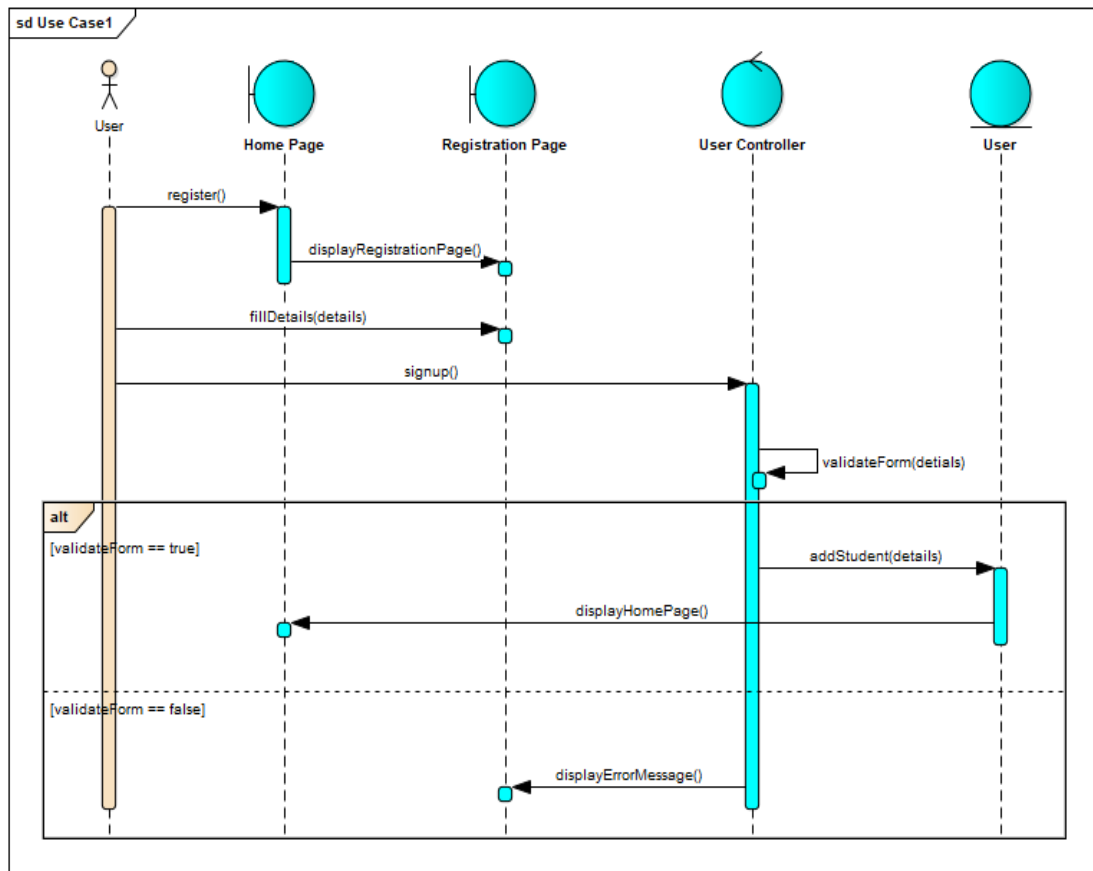


Figure 4.4: Sequence Diagram for < Create Account >

b) SD002: Sequence diagram for Login

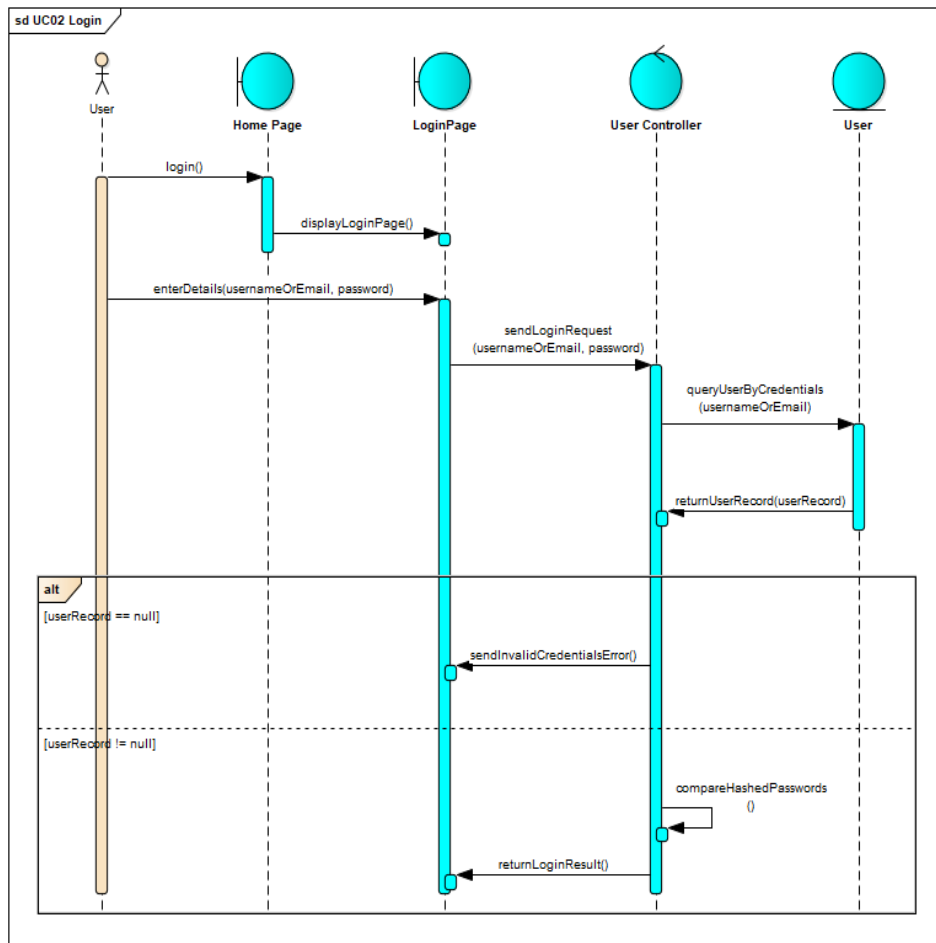


Figure 4.5: Sequence Diagram for < Login >

b) SD003: Sequence diagram for Manage Profile

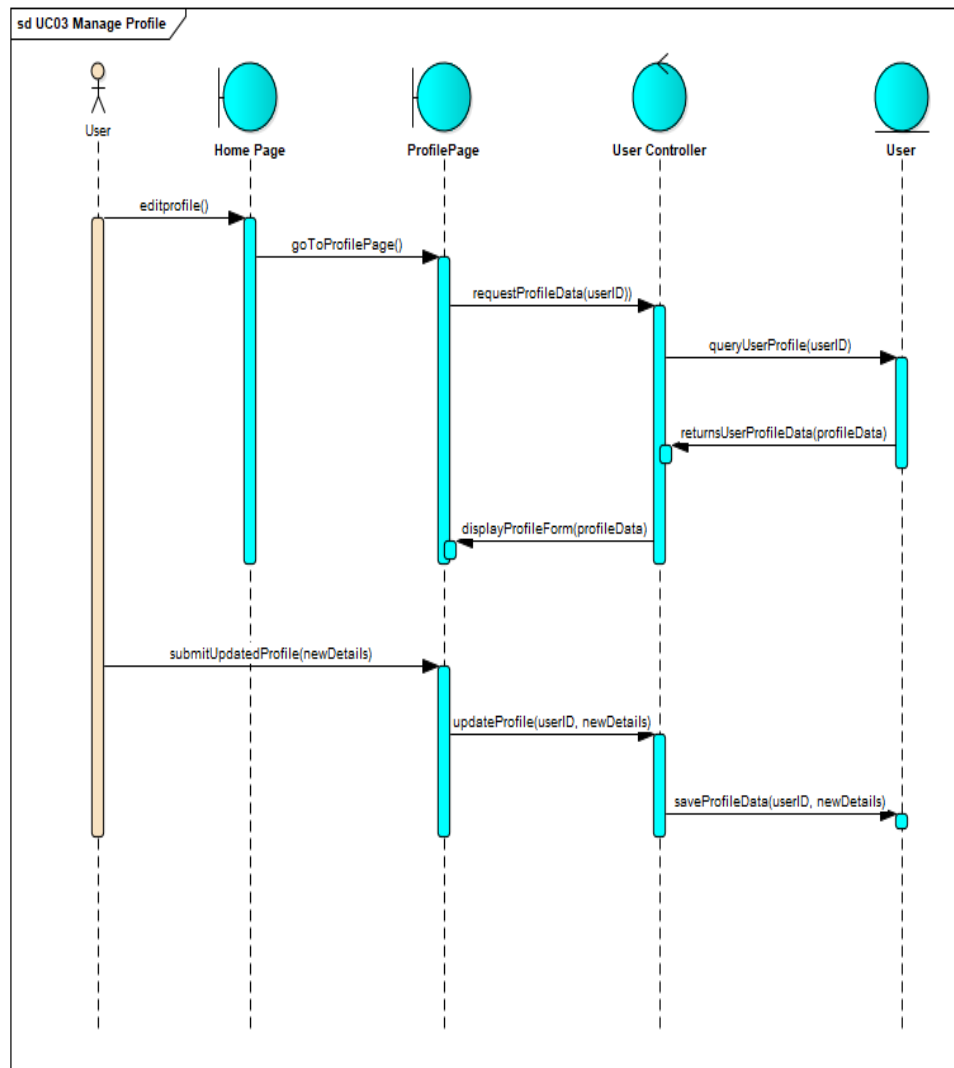


Figure 4.5: Sequence Diagram for < Manage Profile>

b) SD004: Sequence diagram for Process YouTube URL

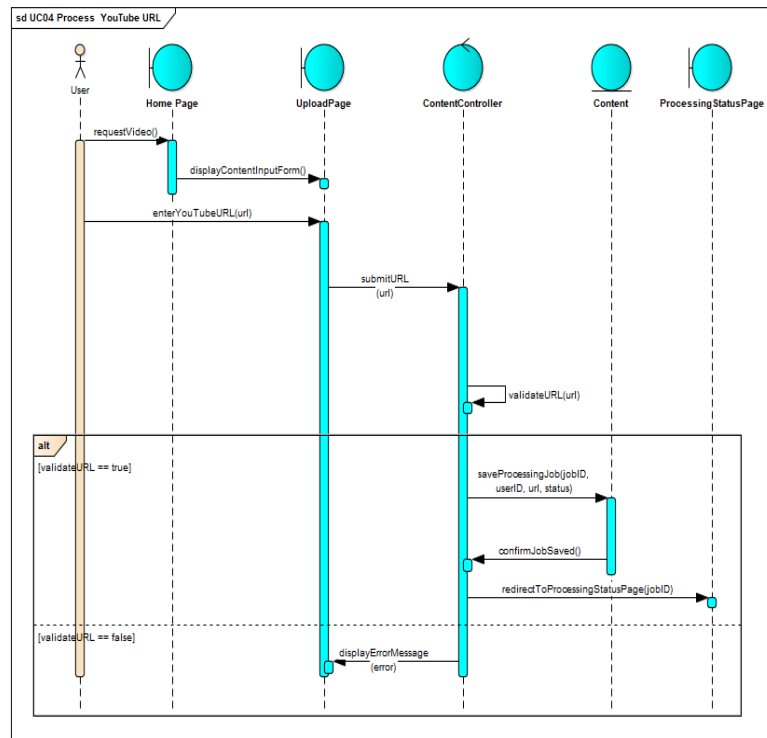


Figure 4.5: Sequence Diagram for < Process YouTube URL>

b) SD005: Sequence diagram for Upload Local Media

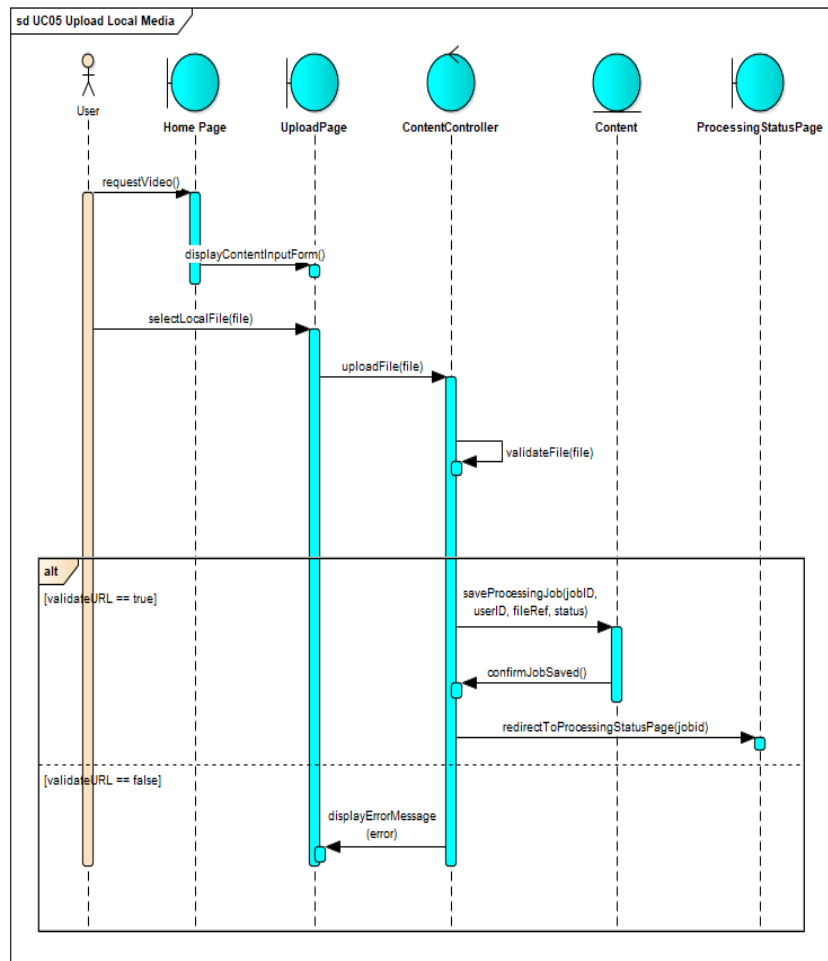


Figure 4.5: Sequence Diagram for < Upload Local Media>

b) SD006: Sequence diagram for Generate Subtitles

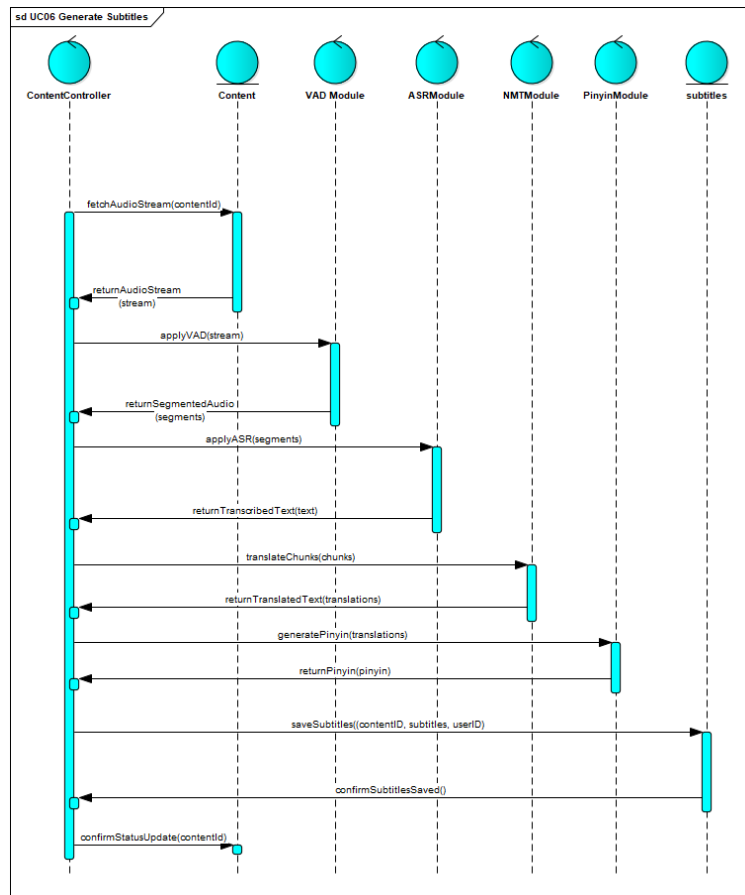


Figure 4.5: Sequence Diagram for < Generate Subtitles>

b) SD007: Sequence diagram for Find Translated Content

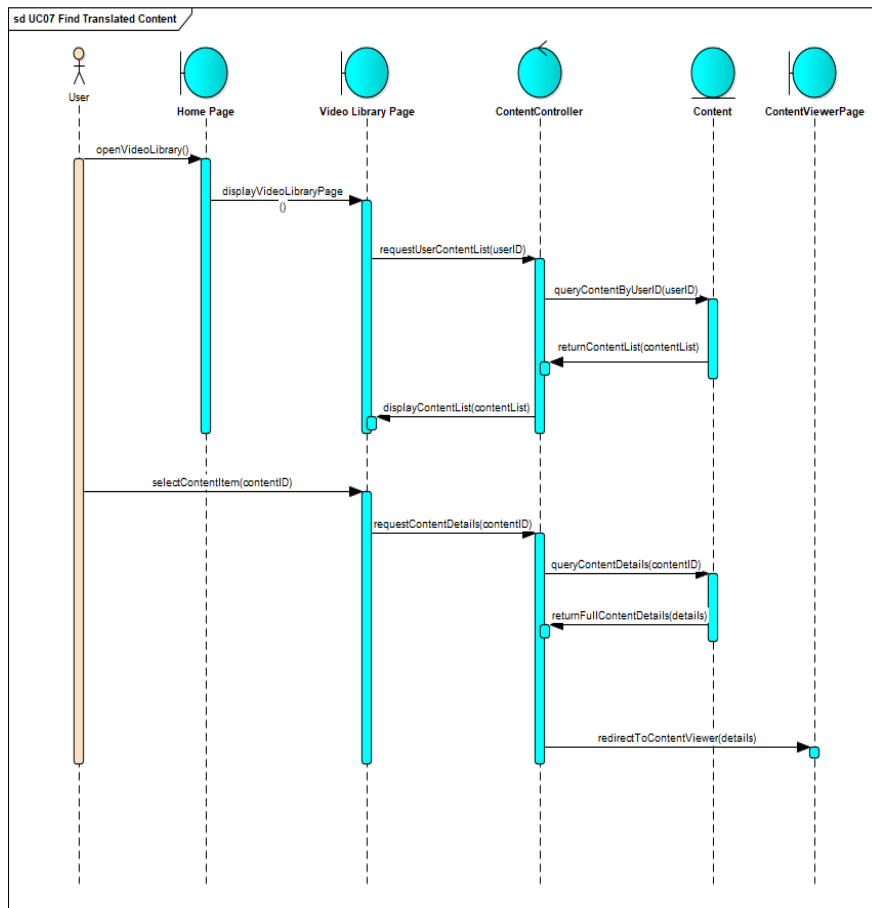


Figure 4.5: Sequence Diagram for < Find Translated Content>

b) SD008: Sequence diagram for Search Word Definition

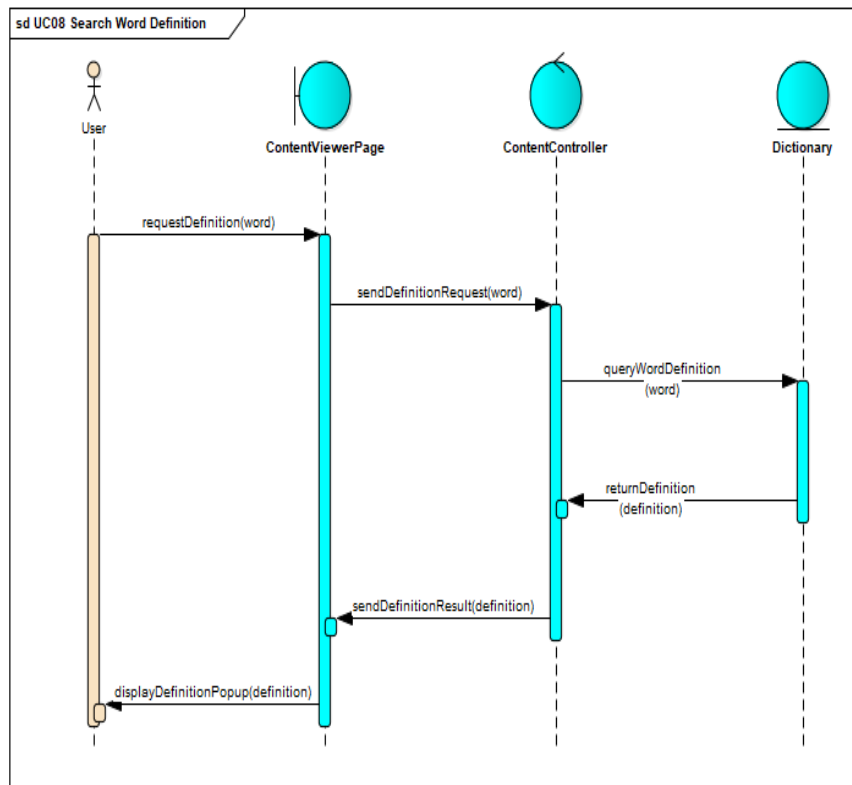


Figure 4.5: Sequence Diagram for < Search Word Definition>

b) SD009: Sequence diagram for Save Vocabulary

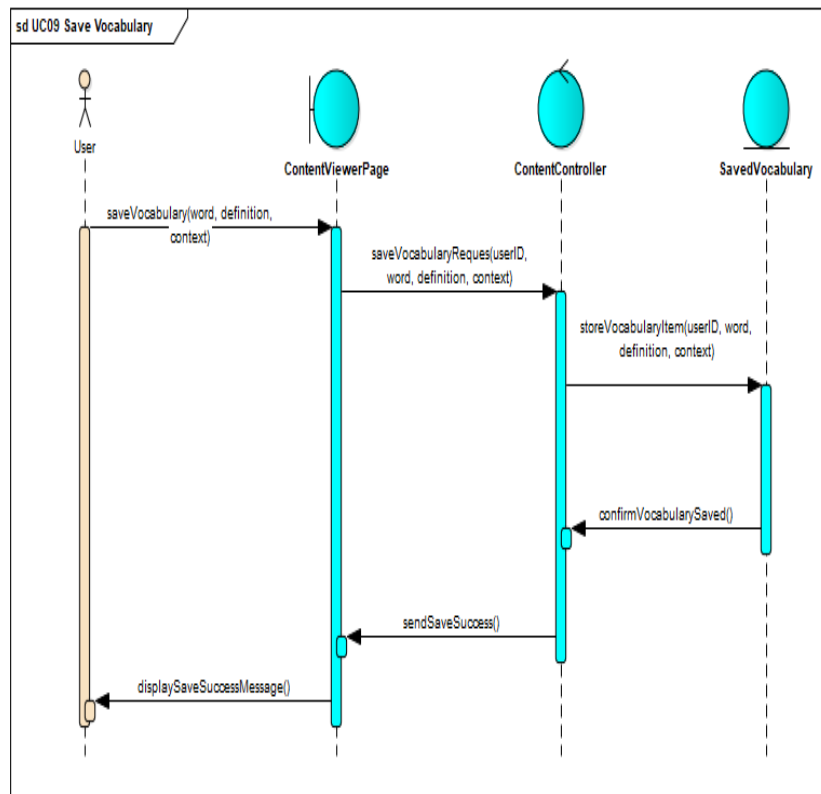


Figure 4.5: Sequence Diagram for < Save Vocabulary>

b) SD010: Sequence diagram for Bookmark Video

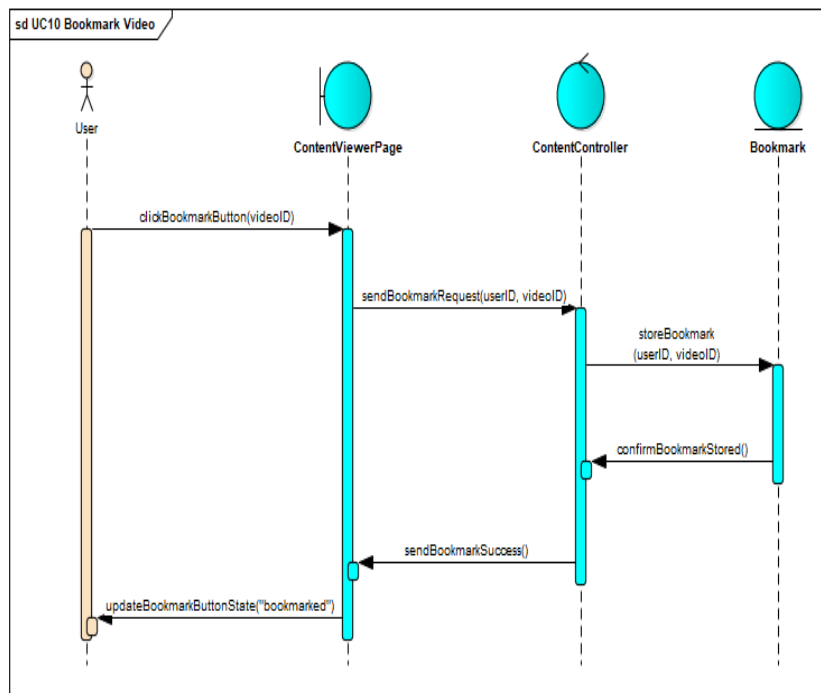


Figure 4.5: Sequence Diagram for < Bookmark Video>

b) SD011: Sequence diagram for Crowd Report Translation Error

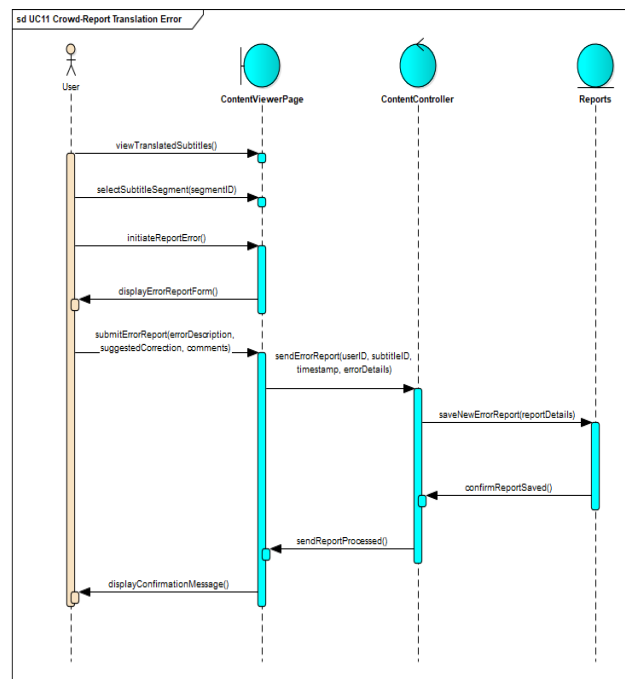


Figure 4.5: Sequence Diagram for < Crowd Report Translation Error>

b) SD012: Sequence diagram for Review Saved Vocabulary

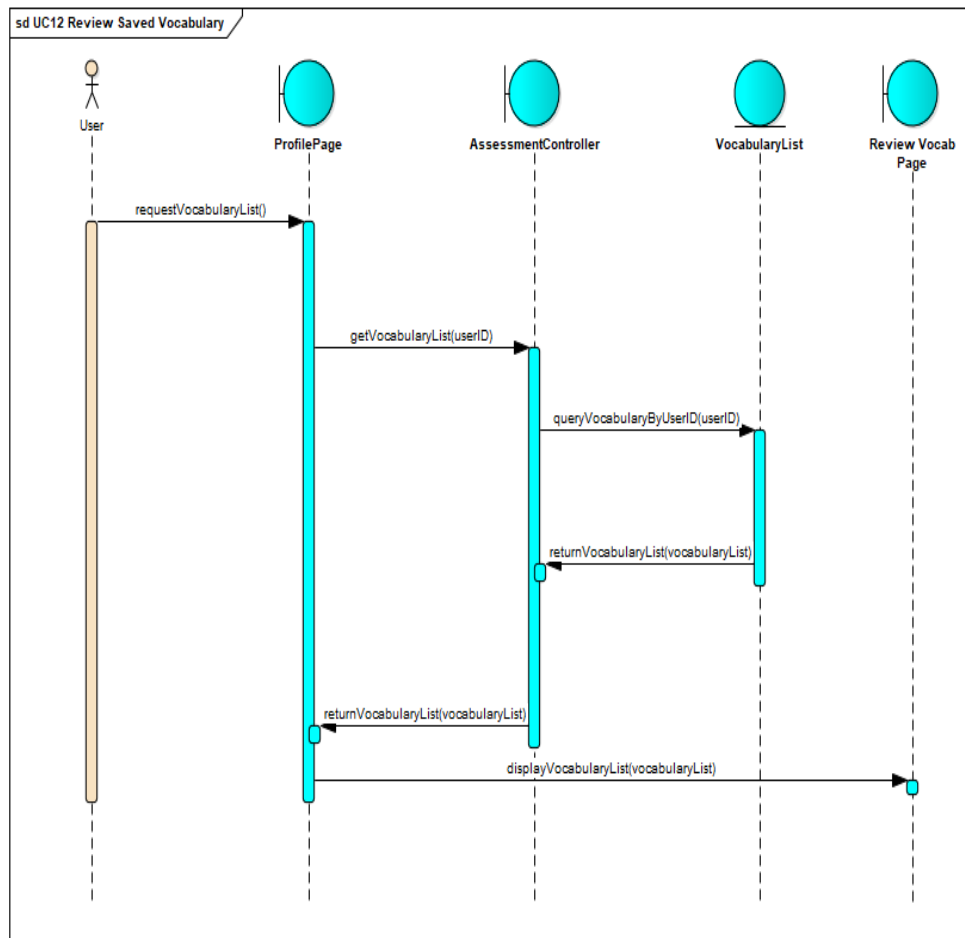


Figure 4.5: Sequence Diagram for < Review Saved Vocabulary>

b) SD013: Sequence diagram for Generate Flashcards

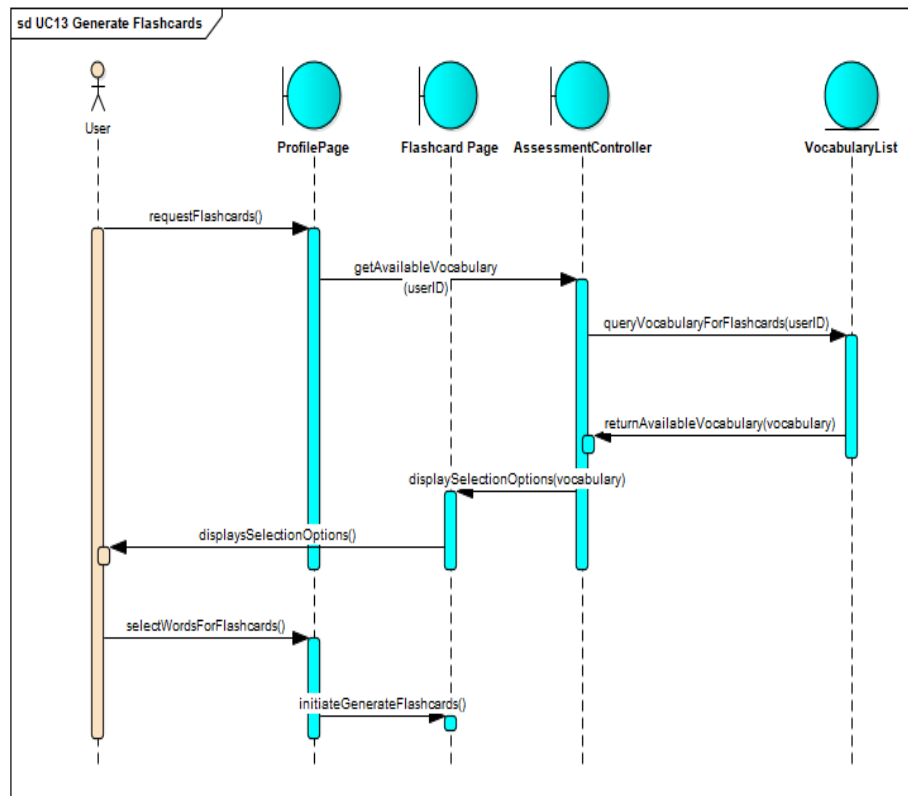


Figure 4.5: Sequence Diagram for < Generate Flashcards>

b) SD014: Sequence diagram for Export Vocabulary List

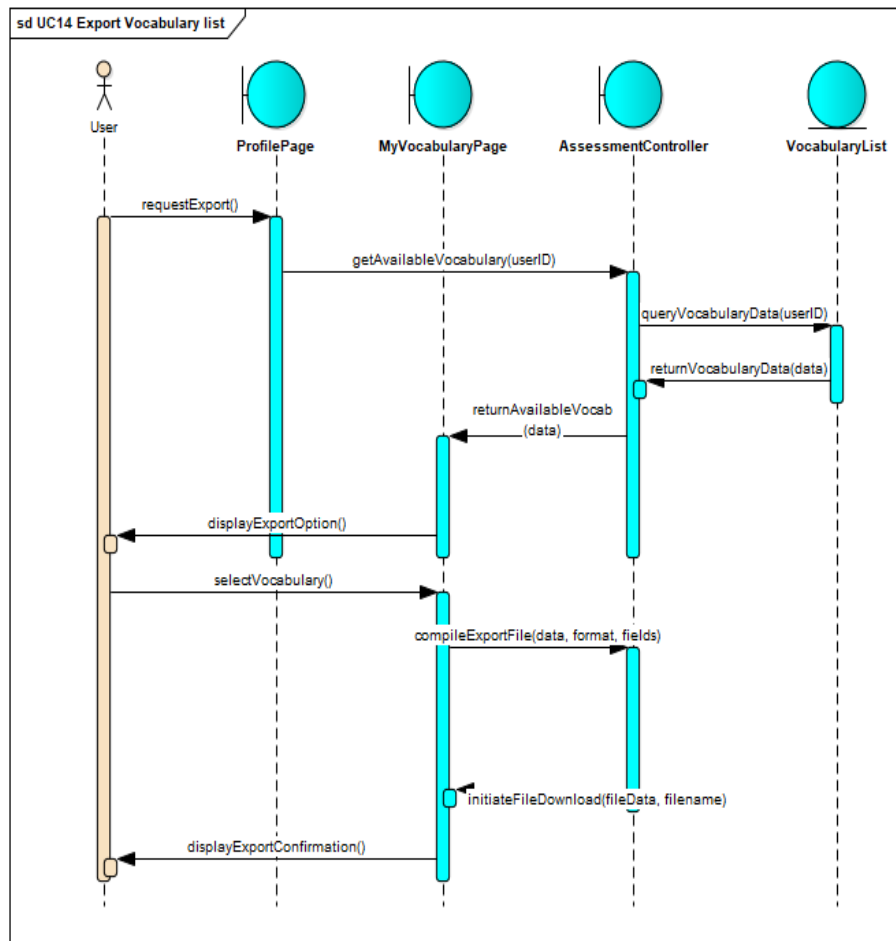


Figure 4.5: Sequence Diagram for < Export Vocabulary List>

b) SD015: Sequence diagram for Generate Quizzes

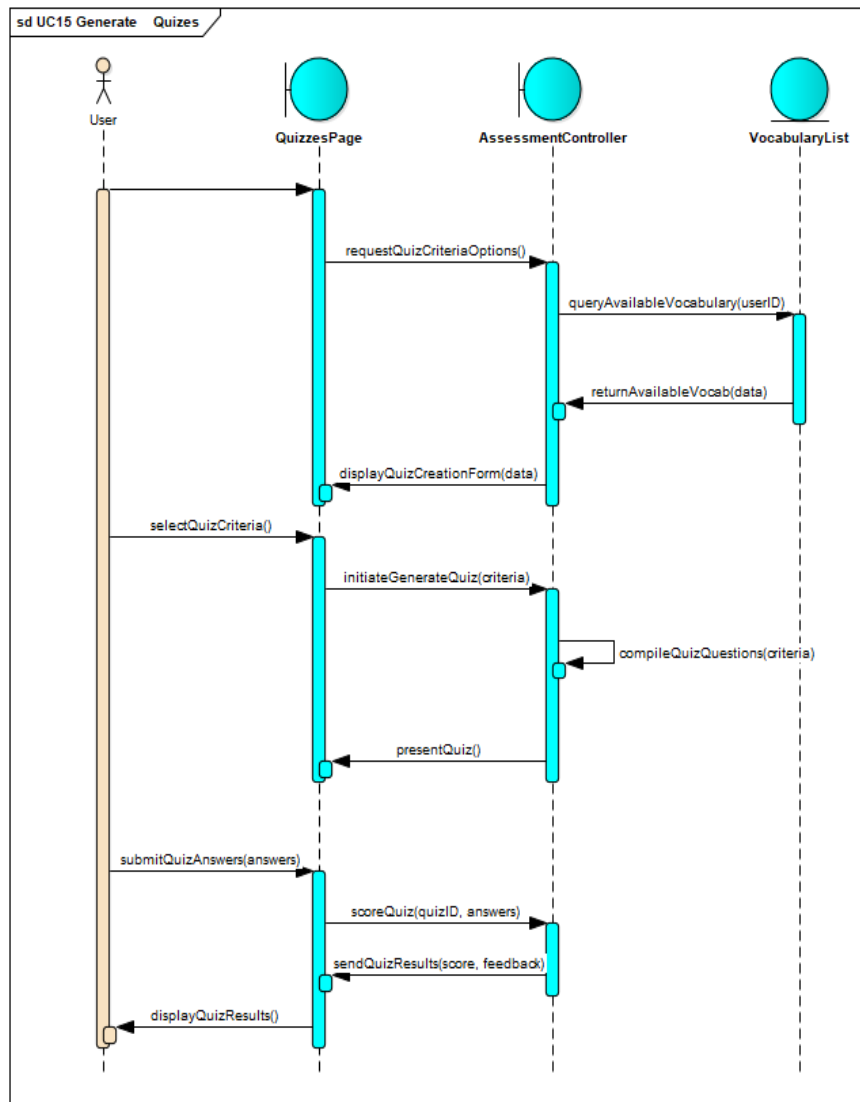


Figure 4.5: Sequence Diagram for < Generate Quizzes>

5.9 State Dynamic Viewpoint

5.9.1 Design Concerns

The State Dynamic Viewpoint focuses on the behavior of the LexiFlow system as it transitions between different operational states in response to user actions or system events. This viewpoint is especially relevant for a reactive system like LexiFlow, where content moves through multiple processing stages triggered by user inputs.

LexiFlow relies on a state-driven content pipeline that manages the lifecycle of uploaded or submitted content — from the initial upload to final subtitle generation. By modeling these states using a state machine, developers and testers can better understand how the system responds under different conditions and ensure robust handling of edge cases or failures.

This viewpoint helps ensure that content moves predictably through the stages of uploaded, segmented, transcribed, translated, annotated, and completed, and that the system can properly resume, retry, or abort when needed.

5.9.2 Design View

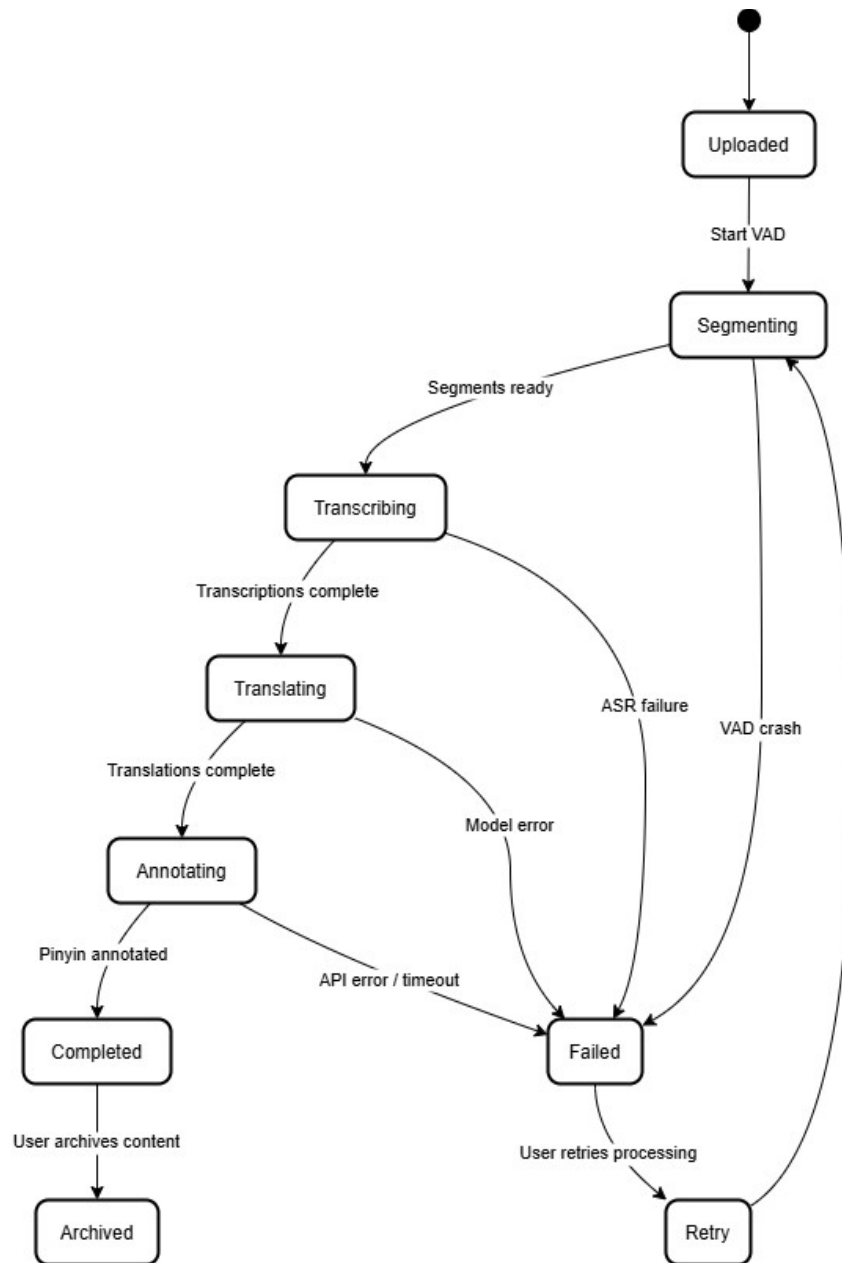


Figure 2.3: State Machine Diagram for Generate Subtitles

5.10 Algorithm Viewpoint

5.10.1 Design Concerns

The Algorithm Viewpoint focuses on the core processing logic inside the LexiFlow system. It provides a clear understanding of how various methods operate, ensuring their correctness, maintainability, and performance. LexiFlow uses several algorithmic routines to process audio/video input, generate bilingual subtitles, and support vocabulary-based learning tools like flashcards and quizzes.

Each processing module — such as transcription, translation, pinyin annotation, and vocabulary handling — is implemented with its own well-defined method. These algorithms also interface with external AI services such as Whisper and SeamlessM4Tv2. To maintain responsiveness, the system ensures that data is processed efficiently and incrementally, minimizing memory load and wait time.

By outlining these steps in detail, developers can test functionality under different scenarios, including edge cases and error conditions, ensuring the system behaves reliably and efficiently in real-world use.

5.10.2 Design View

- a) Algorithm: Subtitle Generation Pipeline
 - Entity: PipelineProcessor
 - Method: generateSubtitles(content: Content): Subtitle[]
 - Input:
 - i. content (uploaded file or YouTube link)
 - Process:
 - i. Extract audio from the content.
 - ii. Apply Silero VAD to detect and split speech into segments.
 - iii. For each segment, transcribe Mandarin speech using Whisper ASR.
 - iv. Translate each transcription to English using SeamlessM4Tv2.
 - v. Annotate Mandarin text with pinyin using PyPinyin.
 - vi. Create Subtitle objects combining timestamp, Mandarin, English, and pinyin.
 - vii. Store subtitles in the database and return them for display.
- b) Algorithm: Flashcard Generation
 - Entity: VocabularyTool
 - Method: generateFlashcards(user: LanguageLearner): Flashcard[]
 - Input:
 - i. user (language learner)

- Process:
 - i. Retrieve the user's saved vocabulary list from the database.
 - ii. For each word, generate a flashcard with the Chinese character, pinyin, and English translation.
 - iii. Shuffle flashcards to randomize order.
 - iv. Store the flashcards in the user's learning progress.
 - v. Return the flashcards for use in interactive review mode.

6. User Interface Design

6.1 Overview of User Interface

The LexiFlow system is designed with a learner-focused and educator-friendly interface to ensure a seamless and intuitive experience for all users. From the moment a user logs in, they are presented with a clean, modern layout that enables quick access to core functionalities such as content upload, subtitle viewing, and vocabulary learning tools. The interface is designed to minimize user confusion, allowing even first-time users to navigate the system with ease.

For language learners, the platform offers an interactive dashboard where they can upload local audio/video files or paste YouTube URLs. Once the content is processed, learners can view synchronized subtitles with Mandarin text, English translation, and pinyin annotations. The subtitle viewer is interactive — users can click on words to view definitions, save vocabulary, or create flashcards and quizzes directly from the content. Bookmarked segments and saved vocabulary are easily accessible for review. The learning experience is further enhanced with tools that track vocabulary progress and enable spaced repetition practice.

For educators, the interface provides access to content creation and learner monitoring tools. Educators can upload curated materials, organize video lessons into thematic playlists, and track student vocabulary growth and content interaction. They also have the ability to review reported translation errors and manage educational resources used in classroom or independent learning contexts.

The interface is fully responsive, adapting smoothly to different screen sizes for both desktop and mobile use. It includes built-in feedback elements such as progress indicators, tooltips, confirmation messages, and error alerts to guide users through their actions. The system ensures users are informed at every stage — from file upload to subtitle generation — promoting transparency and control.

In summary, the LexiFlow user interface is designed to support efficient and enjoyable language learning by offering a clear, structured, and accessible environment for both students and educators. The platform's thoughtful layout and feature accessibility enable users to focus on their learning goals without technical distractions, making LexiFlow a powerful tool for language acquisition.

6.2 Screen Images

LexiFlow

Home Study Contact us About us

EN

Request Video

Insert Youtube Link ...

Cancel Confirm

Fluency is nothing more than mastering the 100 most important real-life conversations

f in t p

2025 LexiFlow. All Rights Reserved.

Terms & Conditions Privacy Policy

Study with Lexiflow
Video Library
Flashcards
Dictionary

Products
Landing page
About us
Contact us

Contact information
+601111382957
info@lexiflow.we
skudai, johor, Malaysia

Request Video Page

Video Library

[Request video](#) 

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

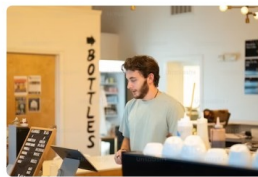
Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)

Youtube video title

HSK Level : 6 views : 198 

[Study Video >](#)[Load More](#)

Fluency is nothing more than mastering the 100 most important real-life conversations



Study with Lexiflow

[Video Library](#)
[Flashcards](#)
[Dictionary](#)

Products

[Landing page](#)
[About us](#)
[Contact us](#)

Contact Information

 +601111382957
 info@lexiflow.we
 skudai, johor, Malaysia

Flashcards

Card 1

中国人喜欢吃.....

Forgot

Easy

Fluency is nothing more than
mastering the 100 most
important real-life
conversations



Study with Lexiflow

[Video Library](#)
[Flashcards](#)
[Dictionary](#)

Products

[Landing page](#)
[About us](#)
[Contact us](#)

Contact information

 +601111382957
 info@lexiflow.we
 skudai, johor, Malaysia

Flashcard front Page

Flashcards

Card 1

中国人喜欢吃米饭。

Next Card

Fluency is nothing more than
mastering the 100 most
important real-life
conversations



Study with Lexiflow

[Video Library](#)
[Flashcards](#)
[Dictionary](#)

Products

[Landing page](#)
[About us](#)
[Contact us](#)

Contact information

 +601111382957
 info@lexiflow.we
 skudai, johor, Malaysia

Flashcard Back Page